Transformers: Contextualizing Word Embeddings

The Modern NLP Paradigm

Firuz Kamalov

MAI 604: Deep Learning

May 1, 2026

Lecture Outline

Outline

A Brief History

- **Pre-2017 Landscape:** NLP was dominated by RNNs and LSTMs.
  - Relied on *static* word embeddings (e.g., Word2Vec).
  - Processed language *sequentially*, step-by-step.

- **Breakthrough:** "Attention Is All You Need" (Vaswani et al., 2017), originally proposed for machine translation.

- **Idea:** Discard recurrences entirely. Process all words in a sequence simultaneously using solely an **attention mechanism**.

- **Paradigm Shift:** This transition from sequential processing to parallel attention solved three major historical bottlenecks, which we will explore next:
  1. Ambiguity & Context
  2. Parallelization & Scalability
  3. Trainability & Gradient Flow

Motivation 1: Contextualized Embeddings & Ambiguity

"The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food."

Challenge of Context:

- Language is highly ambiguous. Who does "it", "they", or "their" refer to?
- Standard fully connected networks or static embeddings fail to capture dynamic meaning.

Transformer Solution:

- Word representations are dynamically updated based on the surrounding sequence.
- The network learns that the word "it" should explicitly "pay attention" to the word "restaurant", creating a highly rich, **contextualized vector**.

Motivation 2: Parallelization and Scalability

- **Sequential Bottleneck:** RNNs and LSTMs process sequences one word at a time. You cannot compute step t until step $t-1$ is finished. This sequential nature severely limits training speed.
- **Parallelization:** Transformers abandon recurrence entirely. They process all words in a sequence simultaneously.
- **Hardware Symbiosis:**
  - Yes, Transformers are computationally massive $O(N^2)$.
  - However, because the operations are entirely parallelizable matrix multiplications, modern GPUs process them with extreme efficiency.
- **Surprising Outcome** (Scaling Laws):
  - Because training became highly parallel, researchers could feed models vastly more data.
  - They discovered that simply increasing model size (more layers/dimensions) and data volume consistently and predictably improves performance.

Motivation 3: Why are they actually easier to train?

1. Shortest Path for Gradients

- In an RNN, for word 100 to affect word 1, the gradient must backpropagate through 100 sequential, non-linear steps. This leads to severe **vanishing gradients**.
- In a Transformer, every word attends to every other word directly. The mathematical path length between *any* two words is exactly **one step**. Gradients flow instantly across the entire sequence.

2. No "Information Bottleneck"

- An RNN must compress the entire history of a paragraph into a single, fixed-size "hidden state" vector.
- A Transformer keeps all word vectors separate and accessible. The attention mechanism retrieves exactly what it needs without forcing a compression bottleneck.

3. Residual Connections & Layer Normalization

- The architecture heavily wraps its attention and MLP blocks in **Residual** connections and **LayerNorm**.
- This creates an extremely smooth loss landscape, allowing us to stack dozens or even hundreds of layers without training collapsing.

Outline

The Core Intuition: Updating Embeddings

- The primary purpose of the transformer is to **update** the original word embedding to reflect its context.
- Simplest Case: Imagine the weight matrices ($\mathbf{\Omega}$) are identity matrices. Then $\mathbf{v}_m = \mathbf{x}_m$.
- The attention mechanism just computes a *weighted average* of the other input vectors in the sentence.
- Residual Connection: We *add* this context-update back to the original word: $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \text{Update}$

Example: *"Apple makes the best phones"*

Let's use 2D vectors: $\begin{bmatrix}\text{Food Feature}\\ \text{Tech Feature}\end{bmatrix}$

1. Initial Embeddings:

$\mathbf{x}_{\text{apple}} = \begin{bmatrix}1\\0\end{bmatrix} \quad \mathbf{x}_{\text{phones}} = \begin{bmatrix}0\\1\end{bmatrix}$

2. The Attention Update:

$\text{sa}_{\text{apple}} = 0.2\mathbf{x}_{\text{apple}} + 0.8\mathbf{x}_{\text{phones}}$

$\text{sa}_{\text{apple}} = 0.2\begin{bmatrix}1\\0\end{bmatrix} + 0.8\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}0.2\\0.8\end{bmatrix}$

3. Transformed Embedding:

$\mathbf{x}_{\text{apple\_new}} = \mathbf{x}_{\text{apple}} + \text{sa}_{\text{apple}}$

$\mathbf{x}_{\text{apple\_new}} = \begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0.2\\0.8\end{bmatrix} = \begin{bmatrix}\mathbf{1.2}\\\mathbf{0.8}\end{bmatrix}$

Result: *The vector for "Apple" has been shifted up the tech axis. It is now representing the company, not the fruit!*

Dot-Product Self-Attention

Step 1: Compute Values
Computed independently for each input:

$$\mathbf{v}_m = \boldsymbol{\beta}_v + \boldsymbol{\Omega}_v \mathbf{x}_m$$

Step 2: Weighted Sum
The n^{th} output is a weighted sum:

$$\text{sa}_n[\mathbf{x}_1, \ldots, \mathbf{x}_N] = \sum_{m=1}^{N} a[\mathbf{x}_m, \mathbf{x}_n]\mathbf{v}_m$$

Numeric Example ($N = 2$ words)

1. Inputs & Parameter Matrix (Let $\boldsymbol{\beta}_v = \mathbf{0}$):

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \boldsymbol{\Omega}_v = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$$

2. Calculate Values ($\mathbf{v}_1, \mathbf{v}_2$):

$$\mathbf{v}_1 = \boldsymbol{\Omega}_v \mathbf{x}_1 = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 10 \\ 0 \end{bmatrix}$$

$$\mathbf{v}_2 = \boldsymbol{\Omega}_v \mathbf{x}_2 = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 10 \end{bmatrix}$$

3. Apply Attention Weights & Sum:
Assume $a[\mathbf{x}_1, \mathbf{x}_1] = 0.8$ and $a[\mathbf{x}_2, \mathbf{x}_1] = 0.2$:

$$\text{sa}_1 = 0.8 \begin{bmatrix} 10 \\ 0 \end{bmatrix} + 0.2 \begin{bmatrix} 0 \\ 10 \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

Self-Attention as Contextual Routing

- The output sa_n is a **weighted average** of all value vectors $\mathbf{v}_m$ in the sequence.

- The attention score $a[\mathbf{x}_m, \mathbf{x}_n]$ dictates what proportion of input m's value is "routed" into output n.

Example:

- *"The restaurant was great, it served good food."*
- To compute the output for $\mathbf{x}_n =$ "it", the model might assign $a[\text{"restaurant"}, \text{"it"}] = 0.8$.
- This heavily routes the properties of "restaurant" into the new representation for "it", resolving its ambiguity.

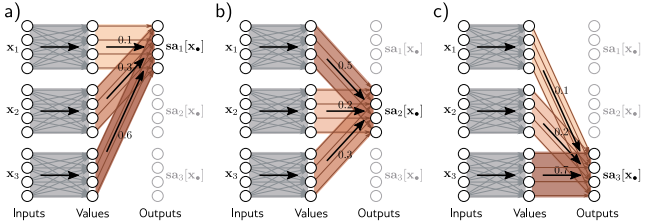

Figure: Routing values: Output 1 (top right) is formed by taking 10% of $\mathbf{v}_1$, 30% of $\mathbf{v}_2$, and 60% of $\mathbf{v}_3$.

Computing and Weighting Values

- The value computation relies on shared parameters $\boldsymbol{\Omega}_v, \boldsymbol{\beta}_v$. This scales linearly with length N.
- It is essentially a sparse matrix operation.
- The attention weights $a[\mathbf{x}_m, \mathbf{x}_n]$ combine values from different inputs.
- The number of attention weights has a quadratic dependence (N^2) on the sequence length N.

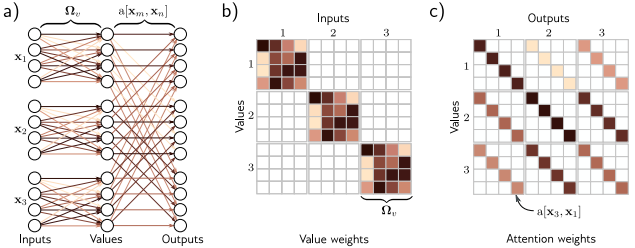

Figure: Matrix sparsity relating inputs to values and values to outputs.

Computing Attention Weights (Keys and Queries)

To compute $a[\mathbf{x}_m, \mathbf{x}_n]$, we first calculate the **queries** and **keys**:

$$\mathbf{q}_n = \boldsymbol{\beta}_q + \boldsymbol{\Omega}_q \mathbf{x}_n$$
$$\mathbf{k}_m = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{x}_m$$

We then take the dot product between queries and keys and apply a **softmax** function:

$$a[\mathbf{x}_m, \mathbf{x}_n] = \frac{\exp[\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^{N} \exp[\mathbf{k}_{m'}^T \mathbf{q}_n]}$$

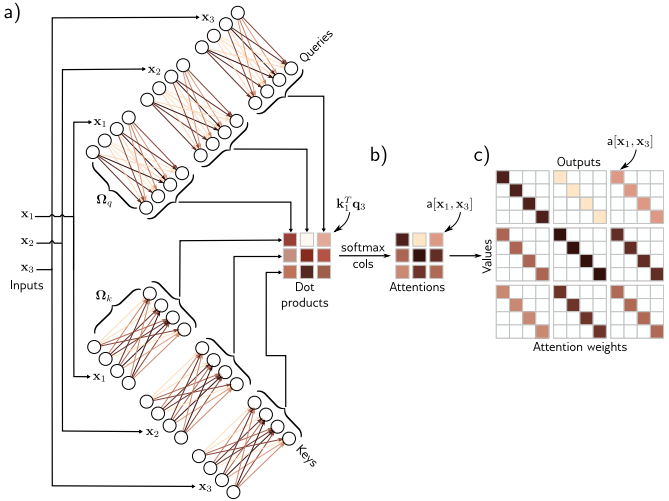

Figure: Computing attention weights via Keys and Queries.

Complete Numeric Example: Self-Attention

1. Inputs & Compute Q, K, V

Inputs & Weights:

$$\mathbf{x}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\mathbf{\Omega}_q = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \mathbf{\Omega}_k = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\mathbf{\Omega}_v = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}$$

2. Attention Scores & Output 1

Complete Numeric Example: Self-Attention

1. Inputs & Compute Q, K, V

Inputs & Weights:

$$\mathbf{x}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\Omega_q = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \Omega_k = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\Omega_v = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}$$

Queries ($\mathbf{q}_n = \Omega_q \mathbf{x}_n$):

$$\mathbf{q}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{q}_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

Keys ($\mathbf{k}_m = \Omega_k \mathbf{x}_m$):

$$\mathbf{k}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{k}_2 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

Values ($\mathbf{v}_m = \Omega_v \mathbf{x}_m$):

$$\mathbf{v}_1 = \begin{bmatrix} 5 \\ -1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

2. Attention Scores & Output 1

Complete Numeric Example: Self-Attention

1. Inputs & Compute Q, K, V

Inputs & Weights:

$$\mathbf{x}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\Omega_q = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \Omega_k = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\Omega_v = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}$$

Queries ($\mathbf{q}_n = \Omega_q \mathbf{x}_n$):

$$\mathbf{q}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{q}_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

Keys ($\mathbf{k}_m = \Omega_k \mathbf{x}_m$):

$$\mathbf{k}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{k}_2 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

Values ($\mathbf{v}_m = \Omega_v \mathbf{x}_m$):

$$\mathbf{v}_1 = \begin{bmatrix} 5 \\ -1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

2. Attention Scores & Output 1

Dot Products (Query 1 vs All Keys):

$$e_{11} = \mathbf{k}_1^T \mathbf{q}_1 = 2$$

$$e_{21} = \mathbf{k}_2^T \mathbf{q}_1 = 1$$

Softmax (Attention Weights):

$$a_{11} = \frac{e^2}{e^2 + e^1} \approx 0.73$$

$$a_{21} = \frac{e^1}{e^2 + e^1} \approx 0.27$$

Final Weighted Sum (Output 1):

$$\text{sa}_1 = a_{11}\mathbf{v}_1 + a_{21}\mathbf{v}_2$$

$$\text{sa}_1 = 0.73 \begin{bmatrix} 5 \\ -1 \end{bmatrix} + 0.27 \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 3.65 \\ -0.73 \end{bmatrix} + \begin{bmatrix} 1.08 \\ 0.27 \end{bmatrix} = \begin{bmatrix} 4.73 \\ -0.46 \end{bmatrix}$$

Self-Attention in Matrix Form

We can stack the N inputs $\mathbf{x}_n$ into a $D \times N$ matrix $\mathbf{X}$. Computations become compact:

$$\mathbf{V}[\mathbf{X}] = \boldsymbol{\beta}_v \mathbf{1}^T + \boldsymbol{\Omega}_v \mathbf{X}$$
$$\mathbf{Q}[\mathbf{X}] = \boldsymbol{\beta}_q \mathbf{1}^T + \boldsymbol{\Omega}_q \mathbf{X}$$
$$\mathbf{K}[\mathbf{X}] = \boldsymbol{\beta}_k \mathbf{1}^T + \boldsymbol{\Omega}_k \mathbf{X}$$

The full dot-product self-attention operation is:

$$\text{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \text{Softmax}\Big[\mathbf{K}[\mathbf{X}]^T \mathbf{Q}[\mathbf{X}]\Big]$$

(Softmax is applied independently to each column).

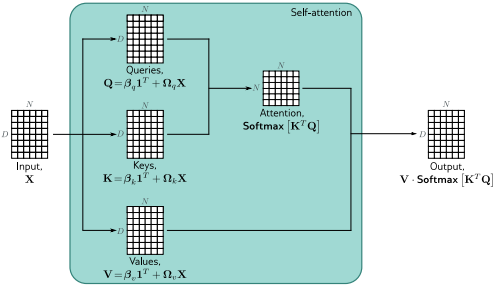

Figure: Self-attention in matrix form.

Outline

1. Positional Encoding

The Problem: Self-attention is *equivariant to permutations*. It inherently ignores word order! *"The woman ate the raccoon"* vs *"The raccoon ate the woman"*.

The Solution: Positional Encodings

- A matrix $\mathbf{\Pi}$ is added to the input embeddings $\mathbf{X}$.
- Each column of $\mathbf{\Pi}$ is unique, allowing the model to distinguish absolute positions in the sequence.

Example Position Matrix ($D = 2, N = 3$)

Suppose we use a simple sine/cosine pattern for $\mathbf{\Pi}$ to encode 3 words:

$$\mathbf{\Pi} = \begin{bmatrix} \sin(1) & \sin(2) & \sin(3) \\ \cos(1) & \cos(2) & \cos(3) \end{bmatrix} = [\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3]$$

We add this to our word embeddings:

$$\mathbf{X}_{\text{new}} = \mathbf{X} + \mathbf{\Pi}$$

Result: Even if word 1 and word 3 are the exact same word ($\mathbf{x}_1 = \mathbf{x}_3$), their inputs into the model ($\mathbf{x}_1 + \mathbf{p}_1 \neq \mathbf{x}_3 + \mathbf{p}_3$) are now uniquely identifiable!

2. Scaled Dot-Product & 3. Multiple Heads

Scaled Dot-Product Self-Attention
Large dot products push the softmax into regions with tiny gradients. We scale by the square root of query/key dimension D_q:

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}\left[\frac{\mathbf{K}^T\mathbf{Q}}{\sqrt{D_q}}\right]$$

Multiple Heads
Apply H attention mechanisms in parallel. Compute sets $\mathbf{V}_h, \mathbf{Q}_h, \mathbf{K}_h$. Their outputs are concatenated and recombined linearly with $\mathbf{\Omega}_c$.

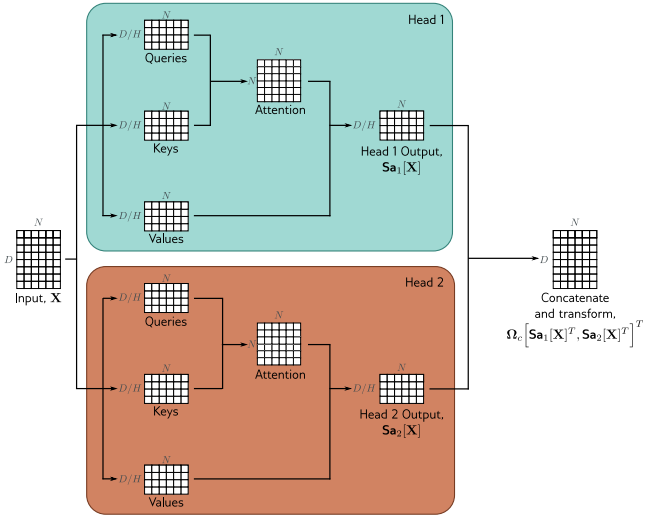

Figure: Multi-head self-attention.

Outline

The Complete Transformer Layer

A full **Transformer Layer** combines:

1. Multi-Head Self-Attention
2. Residual Connections
3. Layer Normalization (LayerNorm)
4. Fully Connected Networks (MLP applied to each word separately)

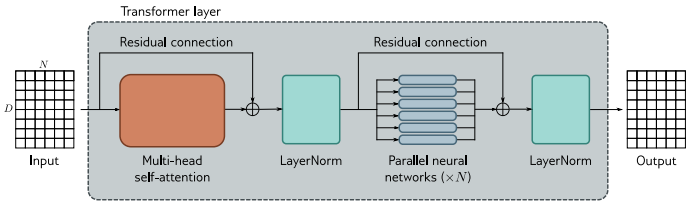

Figure: The operations within a Transformer layer.

Operations in sequence:

$$\mathbf{X} \leftarrow \mathbf{X} + \text{MhSa}[\mathbf{X}]$$
$$\mathbf{X} \leftarrow \text{LayerNorm}[\mathbf{X}]$$
$$\mathbf{x}_n \leftarrow \mathbf{x}_n + \text{mlp}[\mathbf{x}_n] \quad \forall n$$
$$\mathbf{X} \leftarrow \text{LayerNorm}[\mathbf{X}]$$

LayerNorm Example ($D = 4$)

LayerNorm normalizes *across the embedding dimensions* for each word individually. Given a single word embedding $\mathbf{x}_{car} = \begin{bmatrix} 3 & 1 & 5 & -1 \end{bmatrix}^T$:

- **1. Mean:** $\mu = \frac{3+1+5-1}{4} = 2$
- **2. Variance:** $\sigma^2 = \frac{(3-2)^2+(1-2)^2+(5-2)^2+(-1-2)^2}{4} = \frac{20}{4} = 5$
- **3. Normalize:** $\hat{\mathbf{x}} = \frac{\mathbf{x}-\mu}{\sqrt{\sigma^2}} = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 & -1 & 3 & -3 \end{bmatrix}^T$
- **4. Scale & Shift:** Output $= \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta}$

Outline

Transformers for NLP: Tokenization & Embeddings

Tokenization

Text is split using a sub-word tokenizer (e.g., byte pair encoding). Handles unknown words, punctuation, and suffixes by breaking words into manageable sub-word units.

Embeddings

Each token from vocabulary $\mathcal{V}$ maps to a one-hot vector $\mathbf{T}$, then to a dense embedding $\mathbf{X} = \mathbf{\Omega}_e \mathbf{T}$. $\mathbf{\Omega}_e$ is *learned* during training.

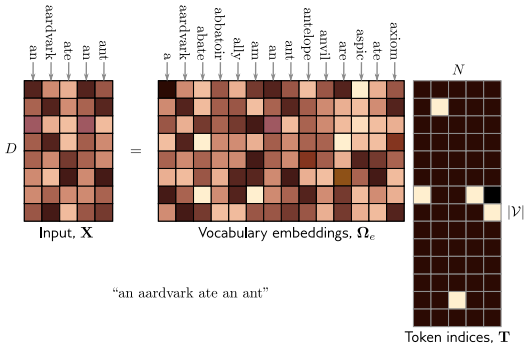

Figure: Input embedding matrix $\mathbf{X}$ formed via $\mathbf{\Omega}_e$ and one-hot tokens.

Encoder Model Example: BERT Pre-Training

BERT is a famous encoder model using stacked Transformer layers. It relies heavily on *Transfer Learning*.

Pre-training Stage

Uses *self-supervision* on massive text corpora.

- A fraction of inputs is replaced with a generic `<mask>` token.
- The model is forced to predict the missing word using bidirectional context.
- Learns syntax, common sense, and rich representations without manual labels.

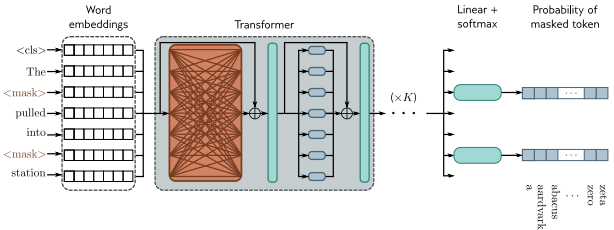

Figure: Masked Language Modeling in BERT.

Encoder Model Example: BERT Fine-Tuning

Fine-Tuning Stage
The pre-trained network is adapted to solve specific tasks using smaller amounts of labeled data.

An extra layer (linear transform or MLP) is appended to convert outputs:

- **Text Classification:** (e.g., Sentiment analysis). Maps the special <cls> token output to a prediction.
- **Word Classification:** (e.g., Named Entity Recognition). Uses the output embedding of each token directly.

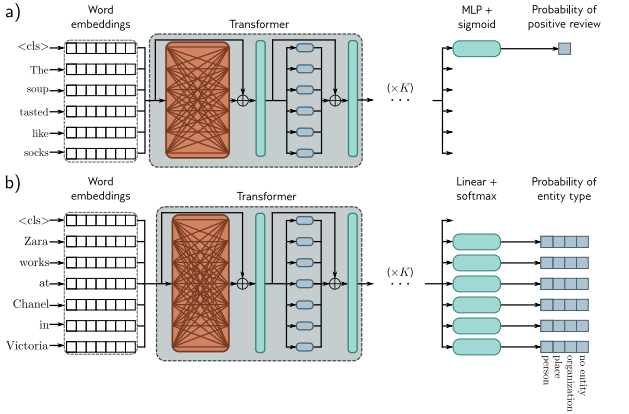

Figure: Fine-tuning the encoder for specific downstream tasks.

Why Doesn't Scaling Lead to Overfitting?

Meta's Llama 3 405B (2024)

- **Parameters:** 405 Billion.
- **Training Data:** 15.6 Trillion tokens.
- **Compute/Cost:** ∼16,000 H100 GPUs for 72 days (est. $50M–$100M+).

Classical ML says 405 billion parameters should catastrophically overfit. Why doesn't it?

1. Data Scales With the Model

- Self-supervision across 15+ trillion tokens provides so much variation that pure memorization becomes mathematically impossible.

2. "Double Descent" Phenomenon

- Deep Learning breaks the classical U-shaped bias-variance tradeoff. In massive overparameterization, SGD tends to settle into "flat," smooth minima that generalize surprisingly well, rather than jagged, overfitted functions.

3. Architectural Regularization

- The architecture itself is highly regularized. Heavy use of Dropout, Weight Decay, LayerNorm, and the dynamic routing of self-attention.

Questions?